

PHP Orientado a Objetos

Lista de Exercícios

Curso Técnico em Informática | 2 horas de aula

Tópicos abordados

Classes, objetos, encapsulamento, construtores, getters e setters

Estrutura da lista

3 questões teóricas · 4 encontre o erro · 8 práticas

ATENÇÃO: O professor estará ausente. Leia os enunciados com calma, use as dicas e consulte o colega antes de desistir.

Parte 1 — Teoria: Encapsulamento

Responda às questões a seguir por escrito. Seja claro e use exemplos quando possível.

1 O que é encapsulamento?

Teoria

Responda com suas próprias palavras:

O que significa encapsular dados em uma classe? Encapsular dados em uma classe é tipo “proteger” as informações de um objeto para que ninguém mexa nelas de qualquer jeito.

Qual é a diferença entre um atributo public, protected e private? A diferença entre **public**, **protected** e **private** é quem pode acessar os atributos ou métodos da classe.

- **public** → qualquer parte do programa pode acessar.
Exemplo: uma variável que todo mundo pode usar sem restrição.
- **protected** → só a própria classe e as classes que herdam dela podem acessar.
Tipo algo “meio protegido”, que não fica liberado pra geral.
- **private** → só a própria classe pode acessar.
É o mais seguro, porque ninguém de fora consegue mexer diretamente.

Por que não é boa prática deixar todos os atributos como public? Porque deixar tudo como **public** faz qualquer parte do programa poder mexer nos dados da classe do jeito que quiser. Isso pode causar vários erros e bagunçar o código.

Dica

Pense em encapsulamento como uma capsula protetora: os dados ficam dentro da classe e so sao acessados por metodos controlados. private = so a propria classe acessa; protected = a classe e suas filhas; public = qualquer lugar do codigo.

2 Para que servem os métodos get e set?

Teoria

Responda:

Por que usamos `getAtributo()` ao invés de acessar `$objeto->atributo` diretamente? Porque o `getAtributo()` dá mais controle e segurança sobre os dados da classe.

O que um método `set` pode fazer além de simplesmente atribuir um valor? Um método `set` não serve só pra colocar um valor no atributo. Ele também pode validar, corrigir ou controlar os dados antes de salvar. Dê um exemplo prático de validação dentro de um `set`. Um exemplo prático é validar uma senha mínima em um cadastro de usuário.

Dica

Um `setIdade($valor)` pode verificar se `$valor > 0` antes de salvar. Isso protege o objeto de receber dados inválidos.

3 Verdadeiro ou Falso — encapsulamento

Teoria

Classifique cada afirmação como Verdadeiro (V) ou Falso (F) e justifique:

"Um atributo `private` pode ser lido diretamente fora da classe."

Falso

Porque atributos `private` só podem ser acessados dentro da própria classe. Fora dela, o acesso direto dá erro. Para ler o valor normalmente usamos um `getter`.

"Getters e setters só fazem sentido quando os atributos são `private`."

Verdadeiro (na prática da programação orientada a objetos)

Porque se o atributo já for `public`, qualquer parte do código consegue acessar direto, então `getters` e `setters` acabam perdendo a principal função, que é controlar o acesso aos dados.

"Encapsulamento aumenta a segurança e a manutenção do código."

Verdadeiro

Porque ele protege os dados da classe e evita alterações erradas. Além disso, deixa o código mais organizado e mais fácil de corrigir ou atualizar depois.

"É possível ter um `getter` sem ter um `setter` para o mesmo atributo."

Verdadeiro

Às vezes a gente quer permitir apenas leitura do valor, mas não alteração. Exemplo: um ID de usuário que não deve mudar depois de criado.

Dica

Lembre-se: atributos `private` só são acessíveis dentro da própria classe. Fora dela, qualquer acesso direto gera erro fatal em PHP.

Parte 2 — Encontre o Erro

Cada questão abaixo contém um código PHP com um erro. Você deve:

Identificar a linha ou trecho com erro

Explicar por que é um erro

Apresentar o código corrigido com comentários

```
<?php
class Produto {
    private $nome;
    private $preco;

    public function __construct($nome, $preco) {
        $this->nome = $nome;
        $this->preco = $preco;
    }
}

$p = new Produto('Teclado', 150.00);
echo $p->nome; // ESTA LINHA TEM ERRO
?>
```

O erro está nesta linha:

```
echo $p->nome;
```

Por que é um erro?

O atributo `$nome` foi declarado como `private`, então ele não pode ser acessado diretamente fora da classe.

Apenas métodos da própria classe conseguem acessar atributos `private`.

Forma correta

O ideal é criar um método getter para retornar o valor do atributo.

Código corrigido:

```
<?php
class Produto {
    private $nome;
    private $preco;

    // Construtor da classe
    public function __construct($nome, $preco) {
        $this->nome = $nome;
        $this->preco = $preco;
    }

    // Getter para acessar o nome do produto
    public function getNome() {
```

```

        return $this->nome;
    }
}

$p = new Produto('Teclado', 150.00);

// Agora o acesso é feito pelo getter
echo $p->getNome();
?>

```

O que foi corrigido?

- Foi criado o método `getNome()`
- O acesso direto ao atributo `private` foi substituído por:

```
$p->getNome()
```

Assim o código respeita o encapsulamento da classe.

Dica

O atributo `$nome` e `private`. Acesso direto fora da classe é proibido em PHP. Crie um método `getNome()` e chame `$p->getNome()`.

5

Erro: uso de variável sem \$this

Encontre o erro

```

<?php
class Aluno {
    private $nome;
    private $nota;

    public function __construct($nome, $nota) {
        $this->nome = $nome;
        $this->nota = $nota;
    }

    public function exibir() {
        echo 'Aluno: ' . $nome . ' - Nota: ' . $nota; // ERRO AQUI
    }
}

$a = new Aluno('Carlos', 8.5);
$a->exibir();
?>

```

O erro está nesta linha:

```
echo 'Aluno: ' . $nome . ' - Nota: ' . $nota;
```

Por que é um erro?

Dentro da classe, os atributos precisam ser acessados usando `$this->`.

As variáveis `$nome` e `$nota` não existem diretamente dentro do método `exibir()`, então o PHP gera erro.

Forma correta

O certo é acessar os atributos assim:

```
$this->nome
```

```
$this->nota
```

Código corrigido:

```
<?php
class Aluno {
    private $nome;
    private $nota;

    // Construtor da classe
    public function __construct($nome, $nota) {
        $this->nome = $nome;
        $this->nota = $nota;
    }

    // Método para exibir os dados do aluno
    public function exibir() {

        // Acesso correto aos atributos usando $this->
        echo 'Aluno: ' . $this->nome . ' - Nota: ' . $this->nota;
    }
}

$a = new Aluno('Carlos', 8.5);

$a->exibir();
?>
```

O que foi corrigido?

Foram trocados:

\$nome

\$nota

por:

\$this->nome

\$this->nota

Porque **\$this** representa o próprio objeto da classe.

Dica

Dentro dos metodos da classe, os atributos devem ser acessados com `$this->nome`, nao simplesmente `$nome` (que seria uma variavel local inexistente).

6

Erro: setter não bloqueia valor inválido

Encontre o
erro

```
<?php
class Pessoa {
    private $idade;

    public function setIdade($idade) {
        if ($idade < 0) {
            echo 'Idade invalida!';
        }
        $this->idade = $idade; // problema: atribui mesmo se invalido
    }

    public function getIdade() {
        return $this->idade;
    }
}

$p = new Pessoa();
$p->setIdade(-5);
echo $p->getIdade(); // imprime -5 !
?>
```

O erro está neste trecho:

```
$this->idade = $idade;
```

Por que é um erro?

Mesmo quando a idade é inválida (menor que 0), o código continua executando e salva o valor no atributo.

Ou seja, a mensagem:

Idade invalida!

aparece, mas a idade **-5** ainda é armazenada no objeto.

Forma correta

O atributo só deve receber valor se ele for válido.

Código corrigido:

```
<?php
class Pessoa {
    private $idade;

    // Método setter com validação
    public function setIdade($idade) {

        // Verifica se a idade é inválida
        if ($idade < 0) {
            echo 'Idade invalida!';
            return; // interrompe o método
        }

        // Só salva se a idade for válida
        $this->idade = $idade;
    }

    // Getter da idade
    public function getIdade() {
        return $this->idade;
    }
}

$p = new Pessoa();

$p->setIdade(-5);

// Agora não imprime -5
echo $p->getIdade();
?>
```

O que foi corrigido?

Foi adicionado:

return;

dentro do **if**.

Assim, quando a idade for inválida, o método para de executar antes de salvar o valor no atributo. Isso impede que dados errados sejam armazenados.

Dica

O if avisa o erro mas não impede a atribuição. Use return após a mensagem de erro, ou coloque a atribuição dentro de um else, para que valores inválidos não sejam salvos.

7

Erro: instanciação sem new

Encontre o erro

```
<?php
class Carro {
    private $modelo;

    public function __construct($modelo) {
        $this->modelo = $modelo;
    }

    public function getModelo() {
        return $this->modelo;
    }
}

$c = Carro('Fusca'); // ERRO NESTA LINHA
echo $c->getModelo();
?>
```

O erro está nesta linha:

```
$c = Carro('Fusca');
```

Por que é um erro?

Para criar um objeto em PHP é obrigatório usar a palavra:

new

Sem o **new**, o PHP entende **Carro()** como se fosse uma função comum, e não uma classe.

Forma correta

O objeto deve ser criado assim:

```
new Carro('Fusca')
```


Código corrigido:

```
<?php
class Carro {
    private $modelo;

    // Construtor da classe
    public function __construct($modelo) {
        $this->modelo = $modelo;
    }

    // Getter do modelo
    public function getModelo() {
        return $this->modelo;
    }
}

// Criação correta do objeto usando "new"
$c = new Carro('Fusca');

// Exibe o modelo do carro
echo $c->getModelo();
?>
```

O que foi corrigido?

Foi adicionada a palavra:

new

na criação do objeto:

```
$c = new Carro('Fusca');
```

Assim o PHP cria corretamente uma instância da classe **Carro**.

Dica

Para criar um objeto em PHP, é obrigatório usar a palavra-chave new: `$c = new Carro('Fusca');`. Sem ela, o PHP trata `Carro()` como uma chamada de função inexistente.

Parte 3 — Exercícios Práticos

Implemente as classes descritas abaixo. Teste cada uma criando objetos e chamando os métodos.

8

Classe Livro

Prático

Crie a classe Livro com os seguintes requisitos:

Atributos privados: \$titulo, \$autor, \$paginas
Construtor que recebe os três valores
Getters e setters para todos os atributos
Método exibir() que imprime todas as informações formatadas
Instancie dois livros diferentes e chame exibir() em ambos
<?php

```
class Livro {

    // Atributos privados
    private $titulo;
    private $autor;
    private $paginas;

    // Construtor
    public function __construct($titulo, $autor, $paginas) {
        $this->titulo = $titulo;
        $this->autor = $autor;
        $this->paginas = $paginas;
    }

    // Getter e Setter do título
    public function getTitulo() {
        return $this->titulo;
    }

    public function setTitulo($titulo) {
        $this->titulo = $titulo;
    }

    // Getter e Setter do autor
    public function getAutor() {
        return $this->autor;
    }

    public function setAutor($autor) {
        $this->autor = $autor;
    }

    // Getter e Setter das páginas
    public function getPaginas() {
        return $this->paginas;
    }

    public function setPaginas($paginas) {
        $this->paginas = $paginas;
    }
}
```

```

// Método para exibir as informações
public function exibir() {
    echo "Título: " . $this->titulo . "<br>";
    echo "Autor: " . $this->autor . "<br>";
    echo "Páginas: " . $this->paginas . "<br><br>";
}
}

// Instanciando dois livros
$livro1 = new Livro("Dom Casmurro", "Machado de Assis", 256);

$livro2 = new Livro("O Pequeno Príncipe", "Antoine de Saint-Exupéry", 96);

// Exibindo informações
$livro1->exibir();

$livro2->exibir();

?>

```

Dica

Estrutura: class Livro { private \$titulo; public function __construct(\$t,\$a,\$p){ \$this->titulo=\$t; ... } }. O método exibir() usa echo com os getters.

9

Classe ContaBancaria

Prático

Crie a classe ContaBancaria:

Atributos privados: \$titular, \$saldo
 Construtor recebe \$titular e inicia \$saldo com 0
 Método depositar(\$valor) — soma ao saldo (validar: valor positivo)
 Método sacar(\$valor) — subtrai do saldo (validar: saldo suficiente)
 Getters para \$saldo e \$titular
 Teste: crie uma conta, deposite R\$500, saque R\$200, exiba o saldo final
 <?php

```

class ContaBancaria {

    // Atributos privados
    private $titular;
    private $saldo;

    // Construtor
    public function __construct($titular) {
        $this->titular = $titular;
        $this->saldo = 0;
    }

    // Método para depositar dinheiro
    public function depositar($valor) {

        // Verifica se o valor é positivo
        if ($valor > 0) {

```

```

        $this->saldo += $valor;
        echo "Depósito de R$ $valor realizado com sucesso!<br>";
    } else {
        echo "Valor inválido para depósito.<br>";
    }
}

// Método para sacar dinheiro
public function sacar($valor) {

    // Verifica se há saldo suficiente
    if ($valor > $this->saldo) {
        echo "Saldo insuficiente.<br>";
    } else {
        $this->saldo -= $valor;
        echo "Saque de R$ $valor realizado com sucesso!<br>";
    }
}

// Getter do titular
public function getTitular() {
    return $this->titular;
}

// Getter do saldo
public function getSaldo() {
    return $this->saldo;
}
}

// Criando uma conta
$conta = new ContaBancaria("Miguel");

// Depositando R$500
$conta->depositar(500);

// Sacando R$200
$conta->sacar(200);

// Exibindo saldo final
echo "<br>Titular: " . $conta->getTitular() . "<br>";
echo "Saldo final: R$ " . $conta->getSaldo();

?>

```

Dica

No metodo sacar, use: `if ($valor > $this->saldo) { echo 'Saldo insuficiente'; return; }` antes de subtrair.

10 Classe Aluno com média

Prático

Crie a classe Aluno:

Atributos privados: \$nome, \$nota1, \$nota2

Construtor recebe os três valores
Getters e setters para todos (notas devem ser entre 0 e 10)
Método calcularMedia() que retorna a média das duas notas
Método situacao() que imprime 'Aprovado' (média >= 5) ou 'Reprovado'

<?php

```
class Aluno {

    // Atributos privados
    private $nome;
    private $nota1;
    private $nota2;

    // Construtor
    public function __construct($nome, $nota1, $nota2) {
        $this->nome = $nome;
        $this->setNota1($nota1);
        $this->setNota2($nota2);
    }

    // Getter e Setter do nome
    public function getNome() {
        return $this->nome;
    }

    public function setNome($nome) {
        $this->nome = $nome;
    }

    // Getter e Setter da nota1
    public function getNota1() {
        return $this->nota1;
    }

    public function setNota1($nota1) {

        // Validação da nota
        if ($nota1 >= 0 && $nota1 <= 10) {
            $this->nota1 = $nota1;
        } else {
            echo "Nota 1 inválida!<br>";
        }
    }

    // Getter e Setter da nota2
    public function getNota2() {
        return $this->nota2;
    }
}
```

```

public function setNota2($nota2) {

    // Validação da nota
    if ($nota2 >= 0 && $nota2 <= 10) {
        $this->nota2 = $nota2;
    } else {
        echo "Nota 2 inválida!<br>";
    }
}

// Método para calcular média
public function calcularMedia() {
    return ($this->nota1 + $this->nota2) / 2;
}

// Método para verificar situação
public function situacao() {

    if ($this->calcularMedia() >= 5) {
        echo "Aprovado";
    } else {
        echo "Reprovado";
    }
}

}

// Teste
$aluno = new Aluno("Carlos", 7, 8);

echo "Aluno: " . $aluno->getNome() . "<br>";
echo "Média: " . $aluno->calcularMedia() . "<br>";
echo "Situação: ";
$aluno->situacao();

?>

```

Dica

calcularMedia() retorna $(\$this->nota1 + \$this->nota2) / 2$. Em situacao(), chame `$this->calcularMedia()` para evitar repetir o calculo.

11 Classe Retangulo

Prático

Crie a classe Retangulo:

- Atributos privados: \$largura e \$altura
- Construtor recebe os dois valores (validar: devem ser positivos)
- Getters e setters para ambos
- Método calcularArea() que retorna largura x altura

Método calcularPerimetro() que retorna 2 x (largura + altura)
Crie dois retângulos e compare qual tem maior área

```
<?php
```

```
class Retangulo {

    // Atributos privados
    private $largura;
    private $altura;

    // Construtor
    public function __construct($largura, $altura) {

        $this->setLargura($largura);
        $this->setAltura($altura);
    }

    // Getter da largura
    public function getLargura() {
        return $this->largura;
    }

    // Setter da largura
    public function setLargura($largura) {

        // Validação
        if ($largura > 0) {
            $this->largura = $largura;
        } else {
            echo "Largura inválida!<br>";
        }
    }

    // Getter da altura
    public function getAltura() {
        return $this->altura;
    }

    // Setter da altura
    public function setAltura($altura) {

        // Validação
        if ($altura > 0) {
            $this->altura = $altura;
        } else {
            echo "Altura inválida!<br>";
        }
    }
}
```

```

// Método para calcular área
public function calcularArea() {
    return $this->largura * $this->altura;
}

// Método para calcular perímetro
public function calcularPerimetro() {
    return 2 * ($this->largura + $this->altura);
}
}

// Criando dois retângulos
$ret1 = new Retangulo(10, 5);

$ret2 = new Retangulo(8, 7);

// Exibindo áreas
echo "Área do Retângulo 1: " . $ret1->calcularArea() . "<br>";

echo "Área do Retângulo 2: " . $ret2->calcularArea() . "<br><br>";

// Comparando áreas
if ($ret1->calcularArea() > $ret2->calcularArea()) {

    echo "O Retângulo 1 possui a maior área.";

} elseif ($ret2->calcularArea() > $ret1->calcularArea()) {

    echo "O Retângulo 2 possui a maior área.";

} else {

    echo "Os dois retângulos possuem a mesma área.";

}

// Exibindo perímetros
echo "<br><br>";
echo "Perímetro do Retângulo 1: " . $ret1->calcularPerimetro() . "<br>";

echo "Perímetro do Retângulo 2: " . $ret2->calcularPerimetro();

?>

```

Dica

Para comparar: `if ($r1->calcularArea() > $r2->calcularArea()) { ... }`. Os métodos são chamados no objeto com `->`.

Crie a classe Funcionario:

Atributos privados: \$nome, \$cargo, \$salario
Construtor recebe os três valores
Getters para todos os atributos
Método aumentarSalario(\$percentual) que aplica o aumento
Método exibir() que mostra nome, cargo e salário formatado
Teste: crie um funcionário, aplique 15% de aumento e exiba antes e depois

<?php

```
class Funcionario {

    // Atributos privados
    private $nome;
    private $cargo;
    private $salario;

    // Construtor
    public function __construct($nome, $cargo, $salario) {
        $this->nome = $nome;
        $this->cargo = $cargo;
        $this->salario = $salario;
    }

    // Getter do nome
    public function getNome() {
        return $this->nome;
    }

    // Getter do cargo
    public function getCargo() {
        return $this->cargo;
    }

    // Getter do salário
    public function getSalario() {
        return $this->salario;
    }

    // Método para aumentar salário
    public function aumentarSalario($percentual) {

        // Calcula o aumento
        $aumento = $this->salario * ($percentual / 100);

        // Soma ao salário atual
```

```

        $this->salario += $aumento;
    }

    // Método para exibir dados
    public function exibir() {

        echo "Nome: " . $this->nome . "<br>";
        echo "Cargo: " . $this->cargo . "<br>";
        echo "Salário: R$ " . number_format($this->salario, 2, ',', '.') . "<br><br>";
    }
}

// Criando funcionário
$f = new Funcionario("Carlos", "Programador", 3000);

// Exibindo antes do aumento
echo "<b>Antes do aumento:</b><br>";
$f->exibir();

// Aplicando aumento de 15%
$f->aumentarSalario(15);

// Exibindo depois do aumento
echo "<b>Depois do aumento:</b><br>";
$f->exibir();

?>

```

Dica

Formula do aumento: $\$this->salario = \$this->salario * (1 + \$percentual / 100)$; Para formatar: `number_format($this->salario, 2, ',', '.')`

13 Classe Temperatura com conversão

Intermediário

Crie a classe Temperatura:

Atributo privado: \$celsius
 Construtor recebe o valor em Celsius
 Getter e setter para \$celsius
 Método paraFahrenheit() — fórmula: $(C \times 9/5) + 32$
 Método paraKelvin() — fórmula: $C + 273.15$
 Exiba as três unidades para: 0°C, 100°C e 37°C

```
<?php
```

```
class Temperatura {
```

```

    // Atributo privado
    private $celsius;

```

```

// Construtor
public function __construct($celsius) {
    $this->celsius = $celsius;
}

// Getter
public function getCelsius() {
    return $this->celsius;
}

// Setter
public function setCelsius($celsius) {
    $this->celsius = $celsius;
}

// Converter para Fahrenheit
public function paraFahrenheit() {
    return ($this->celsius * 9/5) + 32;
}

// Converter para Kelvin
public function paraKelvin() {
    return $this->celsius + 273.15;
}

// Método para exhibir resultados
public function exhibir() {

    echo "Celsius: " . $this->celsius . "°C<br>";

    echo "Fahrenheit: " . $this->paraFahrenheit() . "°F<br>";

    echo "Kelvin: " . $this->paraKelvin() . " K<br><br>";
}
}

// Temperatura 0°C
$temp1 = new Temperatura(0);

// Temperatura 100°C
$temp2 = new Temperatura(100);

// Temperatura 37°C
$temp3 = new Temperatura(37);

// Exhibindo resultados
$temp1->exibir();

```

```
$temp2->exibir();
```

```
$temp3->exibir();
```

```
?>
```

Dica

Os metodos de conversao apenas retornam o valor calculado — eles nao alteram \$celsius.
Use: return (\$this->celsius * 9/5) + 32;

14 Classe Estoque com array de produtos

Intermediário

Crie a classe Estoque:

- Atributo privado: \$produtos (array, inicia vazio)
- Método adicionarProduto(\$nome, \$quantidade) que insere no array
- Método listarProdutos() que exibe todos com um foreach
- Método totalItens() que retorna a soma de todas as quantidades
- Adicione 3 produtos e exiba o estoque completo com o total

```
<?php
```

```
class Estoque {
```

```
    // Atributo privado
```

```
    private $produtos = [];
```

```
    // Método para adicionar produtos
```

```
    public function adicionarProduto($nome, $quantidade) {
```

```
        // Adiciona produto ao array
```

```
        $this->produtos[$nome] = $quantidade;
```

```
    }
```

```
    // Método para listar produtos
```

```
    public function listarProdutos() {
```

```
        echo "<b>Produtos em estoque:</b><br>";
```

```
        // Percorre o array com foreach
```

```
        foreach ($this->produtos as $nome => $quantidade) {
```

```
            echo "Produto: " . $nome . " | Quantidade: " . $quantidade . "<br>";
```

```
        }
```

```
    }
```

```
    // Método para calcular total de itens
```

```
    public function totalItens() {
```

```

$total = 0;

// Soma todas as quantidades
foreach ($this->produtos as $quantidade) {

    $total += $quantidade;
}

return $total;
}
}

// Criando estoque
$estoque = new Estoque();

// Adicionando produtos
$estoque->adicionarProduto("Teclado", 10);

$estoque->adicionarProduto("Mouse", 15);

$estoque->adicionarProduto("Monitor", 5);

// Exibindo estoque
$estoque->listarProdutos();

// Exibindo total de itens
echo "<br><b>Total de itens:</b> " . $estoque->totalItens();

?>

```

Dica

Para adicionar: `$this->produtos[] = ['nome'=>$nome,'qtd'=>$quantidade];` No foreach: `foreach($this->produtos as $p){ echo $p['nome'].': '.$p['qtd']; }`

15 Classe Agenda de contatos (desafio)

Desafio

Una dois conceitos: use a classe Contato dentro de Agenda.

Classe Contato: atributos privados \$nome e \$telefone, construtor e getters
 Classe Agenda: atributo privado \$contatos (array de objetos Contato)
 Método adicionarContato(\$nome, \$telefone) que cria new Contato e insere no array
 Método listar() que percorre o array e exhibe cada contato
 Adicione 3 contatos e liste a agenda

```
<?php
```

```
// Classe Contato
```

```

class Contato {

    // Atributos privados
    private $nome;
    private $telefone;

    // Construtor
    public function __construct($nome, $telefone) {
        $this->nome = $nome;
        $this->telefone = $telefone;
    }

    // Getter do nome
    public function getNome() {
        return $this->nome;
    }

    // Getter do telefone
    public function getTelefone() {
        return $this->telefone;
    }
}

// Classe Agenda
class Agenda {

    // Array de objetos Contato
    private $contatos = [];

    // Método para adicionar contato
    public function adicionarContato($nome, $telefone) {

        // Cria novo objeto Contato
        $contato = new Contato($nome, $telefone);

        // Adiciona ao array
        $this->contatos[] = $contato;
    }

    // Método para listar contatos
    public function listar() {

        echo "<b>Agenda de Contatos:</b><br><br>";

        // Percorre o array de objetos
        foreach ($this->contatos as $contato) {

            echo "Nome: " . $contato->getNome() . "<br>";
        }
    }
}

```

```

        echo "Telefone: " . $contato->getTelefone() . "<br><br>";
    }
}
}

// Criando agenda
$agenda = new Agenda();

// Adicionando contatos
$agenda->adicionarContato("Carlos", "(11) 99999-1111");

$agenda->adicionarContato("Ana", "(11) 98888-2222");

$agenda->adicionarContato("Pedro", "(11) 97777-3333");

// Listando agenda
$agenda->listar();

?>

```

Dica

Em adicionarContato: `$c = new Contato($nome, $telefone); $this->contatos[] = $c;` Em listar: acesse com `$c->getNome()` e `$c->getTelefone()`.

Bom trabalho! Em caso de dúvida, consulte o material da aula ou converse com os colegas.